

**Informe de Actividad 4: Despliegue en producción, métricas de sostenibilidad y gobernanza
de datos**

Deymar Lozano Carballo

Universidad Privada Domingo Savio (U.P.D.S.)

Programación Web II

Jimmy Nataniel Requena Llorentty

29 de septiembre de 2026

Resumen

En la Actividad 4 del proyecto *Feria de Agricultura Familiar* se llevó a cabo el despliegue de la aplicación full-stack (React + Flask + PostgreSQL) a un entorno de producción accesible públicamente, la implementación de un tablero de métricas de sostenibilidad ambiental y la redacción de un plan de gobernanza de datos y seguridad. El despliegue se realizó sobre la plataforma **Vercel** tras descartar **Render** por la exigencia de una tarjeta de pago no disponible en el plan gratuito. La aplicación quedó disponible en Internet en un solo runtime Python que sirve API (/api/*), documentación Swagger (/apidocs) y el frontend compilado (dist/) con *fallback* SPA. Se añadió el endpoint GET /api/metricas, que calcula en tiempo real la huella de carbono por vista (0,62 g CO₂e), el volumen del catálogo y el impacto evitado por compra local (16,8 kg CO₂e). Se construyó una suite de **33 pruebas e2e** (100 % aprobadas) que verifican autenticación, roles, CRUD y métricas. El informe documenta la metodología, los resultados y las buenas prácticas de gobernanza adoptadas.

****Palabras clave:**** despliegue, sostenibilidad, huella de carbono web, Flask, React, PostgreSQL, Vercel, gobernanza de datos.

Índice general

Resumen.....	2
1. Introducción	5
2. Objetivos	5
2.1. Objetivo general.....	5
2.2. Objetivos específicos	6
3. Marco teórico	6
3.1. Arquitectura de aplicaciones full-stack.....	6
3.2. Despliegue *serverless* y plataformas PaaS	6
3.3. Sostenibilidad digital y huella de carbono web	7
3.4. Seguridad en aplicaciones web	7
3.5. Normativa boliviana aplicable	7
4. Metodología	8
5. Desarrollo.....	8
5.1. Unificación del despliegue todo-en-uno	8
5.2. Selección de la plataforma de despliegue	9
5.3. Tablero de métricas de sostenibilidad.....	10
5.4. Pruebas automatizadas	10
5.5. Gobernanza de datos	10
6. Resultados	11
7. Discusión.....	11
8. Conclusiones	12
9. Recomendaciones	12
10. Referencias.....	13
Anexo A — Matriz de uso de inteligencia artificial	14
Anexo B — Evidencia de despliegue y verificación	15

Índice de tablas

Tabla 1. Fases del proceso metodológico y su verificación.....	8
Tabla 2. Bloques de métricas de sostenibilidad con valores verificados	10
Tabla 3. Indicadores clave de la Actividad 4.....	11
Tabla 4. Matriz de uso de asistencia de inteligencia artificial y verificación	14
Tabla 5. Verificación del sitio publicado (checklist de producción)	15

Informe de Actividad 4: Despliegue en producción, métricas de sostenibilidad y gobernanza de datos

1. Introducción

La Feria de Agricultura Familiar es una plataforma web que conecta a pequeños productores agroecológicos con consumidores urbanos. En las actividades anteriores se desarrolló el backend en Flask con PostgreSQL y autenticación JWT (roles *admin* y *producer*), y el frontend en React con Vite, separado del servicio API. Hasta la Actividad 3 el sistema solo funcionaba en entorno local.

La Actividad 4 persigue tres metas: **(a)** publicar la aplicación en Internet para que usuarios reales puedan acceder desde cualquier dispositivo; **(b)** medir y hacer visible el impacto ambiental del sitio como aporte a la *sostenibilidad digital*; y **(c)** formalizar un plan de gobernanza que garantice seguridad, disponibilidad y tratamiento responsable de los datos.

El presente informe da cuenta de cada fase: selección de la plataforma de despliegue, configuración del entorno de producción, implementación del tablero de sostenibilidad, ampliación de pruebas automatizadas, elaboración del plan de gobernanza y verificación final del sitio publicado.

2. Objetivos

2.1. Objetivo general

Desplegar en producción la plataforma Feria de Agricultura Familiar, incorporar un tablero de métricas de sostenibilidad ambiental y establecer un plan de gobernanza de datos y seguridad que asegure su operación confiable.

2.2. Objetivos específicos

1. Publicar la aplicación completa (API + frontend) en una plataforma *serverless* accesible por HTTPS.
2. Implementar el endpoint GET /api/metricas y la página /sostenibilidad con la huella de carbono por visita y el impacto evitado por la compra local.
3. Ampliar la suite de pruebas e2e a 33 casos (incluyendo métricas) y dejar un pipeline local de verificación (lint + tests) previo a cada despliegue.
4. Redactar docs/GOBERNANZA.md con roles, ciclo de vida de datos, seguridad OWASP, monitorización, respuesta a incidentes y despliegue.
5. Verificar la aplicación desplegada mediante un checklist de producción (salud, catálogo, autenticación, Swagger y tablero).

3. Marco teórico

3.1. Arquitectura de aplicaciones full-stack

Una aplicación full-stack combina un *frontend* (interfaz de usuario) con un *backend* (lógica y datos). En este proyecto el frontend es una SPA construida con React y Vite, y el backend una API REST en Flask que gestiona una base de datos PostgreSQL mediante psycopg. La comunicación entre ambos se realiza con JSON sobre HTTP(S).

3.2. Despliegue **serverless** y plataformas PaaS

Las plataformas como Render y Vercel permiten desplegar aplicaciones sin administrar servidores. Vercel ofrece un **runtime Python** que detecta una aplicación WSGI (Flask) a través de una variable de nivel superior app y reescribe las rutas mediante vercel.json; las dependencias se instalan desde el requirements.txt de la raíz del proyecto (Vercel, 2026). Entre sus ventajas

están HTTPS automático, integración continua con GitHub, *rollback* con un clic y un plan gratuito sin tarjeta de crédito.

3.3. Sostenibilidad digital y huella de carbono web

La *sostenibilidad digital* busca reducir el impacto ambiental de los servicios informáticos. Una métrica popular es el **CO₂e emitido por cada visita** a una página web, estimado con la metodología *Website Carbon* (Greenwood et al., 2020), que combina el peso de los datos transmitidos y la intensidad de carbono de la energía que alimenta el data center:

$$\text{gCO}_2\text{e por vista} = (\text{bytes transferidos} \times 1,9 \times 0,000000151) + 0,522$$

El factor 1,9 considera la descarga real frente al peso del archivo; 0,000000151 convierte la cantidad de datos en emisiones, y 0,522 suma la energía promedio que gasta un data center por visita (Website Carbon, 2025).

3.4. Seguridad en aplicaciones web

El modelo **OWASP ASVS** organiza los requisitos de seguridad de una aplicación en niveles de verificación (autenticación, control de acceso, inyección, configuraciones, etc.). Las mitigaciones clave del sistema son: consultas parametrizadas contra inyección SQL, JWT con expiración para sesiones, roles firmados en el token, CORS restringido y secretos en variables de entorno.

3.5. Normativa boliviana aplicable

La **Ley N.º 164 de Telecomunicaciones, Tecnologías de Información y Comunicación** (Asamblea Legislativa del Estado Plurinacional de Bolivia, 2011) y su Reglamento de Ciberseguridad disponen la protección de la infraestructura de información, la gestión de incidentes y el tratamiento responsable de datos personales. Adicionalmente, la Constitución

Política del Estado reconoce el derecho a la intimidad (art. 21), base del tratamiento prudente de los datos de productores y consumidores.

4. Metodología

Se utilizó una metodología iterativa en siete fases, siempre con verificación previa a cada avance:

Tabla 1

Fases del proceso metodológico y su verificación

Fase	Actividad	Verificación
0	Inicializar git, configurar .gitignore	git status limpio; no se versionan secretos
1	Unificar API + frontend en un único servicio (Flask sirve dist/)	Pruebas locales 5/5 (home, SPA, health, ferias, 404 JSON)
2	Cambio de plataforma: Render → Vercel (imposibilidad de tarjeta)	Import del repositorio; api/index.py + vercel.json
3	Endpoint /api/metricas y página /sostenibilidad	curl local + revisión visual del tablero
4	Suite e2e ampliada (33 casos) y lint	33/33 PASS; npm run lint sin errores
5	Commit + push a GitHub (integración continua)	origin/main = HEAD; despliegue automático
6	Plan de gobernanza (docs/GOBERNANZA.md)	Revisión por secciones y checklist
7	Informe final y guion de presentación	Verificación del checklist de producción

Nota. Elaboración propia a partir de los datos obtenidos del sistema en producción.

Herramientas: Python 3.10, Flask 3.1, PostgreSQL (Supabase), Node.js 22/Vite, React 19, flasgger (OpenAPI), PyJWT, Vercel, GitHub.

5. Desarrollo

5.1. Unificación del despliegue todo-en-uno

Para que una sola aplicación sirviera la API y el frontend:

- `src/services/feriasApi.js` pasó a usar `API_URL = import.meta.env.VITE_API_URL || '/api'`, y `vite.config.js` agregó un *proxy* de desarrollo (`/api` → `http://localhost:5000`). Así el mismo código funciona en local y en producción.
- `backend/app/__init__.py` sirve la carpeta `dist/` (build de Vite) y redirige las rutas que no son `/api/*` al `index.html` (*fallback* SPA), con manejo de errores que devuelve JSON en la API.
- `dist/` se versiona en el repositorio para que la plataforma hospede exactamente el frontend compilado.

Resultado local verificado: `/` (200 HTML), `/ingresar` (200 SPA), `/api/health` (ok), `/api/ferias` (3 ferias), y ruta API inexistente (404 JSON).

5.2. Selección de la plataforma de despliegue

Se preparó primero **Render** (`render.yaml` + `render-build.sh`). No obstante, Render exige registrar una tarjeta de pago incluso en su plan gratuito y la única tarjeta disponible fue rechazada por el proveedor. Se migró a **Vercel**, que:

- ofrece plan *Hobby* **sin tarjeta** y con integración directa con GitHub;
- detecta el framework mediante `framework`: el runtime Python reconoce app en `api/index.py`;
- aplica el *rewrite* `"/(.*)"` → `"/api/index"` (ver `vercel.json`);
- instala dependencias desde `requirements.txt` de la **raíz** del repositorio.

Variables de entorno cargadas: `DATABASE_URL` (Supabase), `SECRET_KEY`, `JWT_SECRET`, `JWT_EXPIRES_MINUTES=60`, `FLASK_ENV=production` y `CORS_ORIGINS=https://feria-agricultura-familiar.vercel.app`.

5.3. Tablero de métricas de sostenibilidad

Se creó el blueprint backend/app/blueprints/metricas.py que expone GET /api/metricas con tres bloques:

Tabla 2

Bloques de métricas de sostenibilidad con valores verificados

Bloque	Métricas	Valor verificado
técnico	tamano_kb (peso de dist/), num_archivos, co2_por_vista_g	348,4 KB · 8 archivos · 0,62 g CO₂e/vista (medido en producción)
catálogo	ferias, productores, productos, destacadas, próximas, ubicaciones	3 · 2 · 11 · 3 · 3 · 6
social	productos comercializados, km evitados, CO ₂ evitado	11 · 105 km · 16,8 kg CO₂e

Nota. Elaboración propia a partir de los datos obtenidos del sistema en producción.

El bloque metodología documenta las constantes: 35 km por feria local, 0,16 kg CO₂e por km de transporte urbano evitado y la fórmula de *Website Carbon*. La página src/pages/Sostenibilidad.jsx (ruta /sostenibilidad) consume el endpoint y presenta los datos; el endpoint quedó además documentado en **Swagger** (/apidocs/web).

5.4. Pruebas automatizadas

backend/scripts/pruebas_e2e.py consolida la suite de extremo a extremo: autenticación, roles (productor vs. admin), CRUD de ferias/productos y las nuevas pruebas de métricas. Ejecución: **33/33 PASS**, con limpieza automática de los datos de prueba. El lint (oxlint) se configuró para ignorar dist/ y no genera advertencias.

5.5. Gobernanza de datos

docs/GOBERNANZA.md define roles y acceso de privilegio mínimo, ciclo de vida del dato (recolección → almacenamiento → uso → respaldo → eliminación), controles OWASP, normativa boliviana (Ley 164), monitorización con health-check cada 5 minutos, matriz de

incidentes con SLA, gestión de cambios con *rollback* en Vercel, accesibilidad WCAG 2.1 y política de sostenibilidad digital (límite de +20 % de peso por release).

6. Resultados

1. **Producción:** la aplicación está publicada en <https://feria-agricultura-familiar.vercel.app>, servida íntegramente por HTTPS; API y SPA conviven en un mismo runtime Python.
2. **Métricas de sostenibilidad:** `/api/metricas` responde con datos reales y documentados; la huella estimada es **0,62 g CO₂e por visita**, y el catálogo local evita **16,8 kg CO₂e** por los 105 km no recorridos por transporte urbano en compras de feria.
3. **Pruebas:** 33/33 casos e2e aprobados; lint sin errores; revisión manual del flujo completo (registro → login → alta de producto → alta de feria) sin incidencias.
4. **Gobernanza:** documento de gobernanza y seguridad completo y enlazado con la operación real (health-check, respaldos diarios de Supabase, rollback en Vercel).
5. **Repositorio integrado:** GitHub como fuente única de verdad con despliegue automático en cada *push* a main.

Tabla 3

Indicadores clave de la Actividad 4

Indicador	Valor
Peso del frontend compilado	348,4 KB (JS 300,9 KB + CSS 29,6 KB) medido en producción
Huella por vista estimada	0,62 g CO ₂ e
Pruebas automatizadas	33/33 (100 %)
Rutas de la API documentadas en Swagger	7
Plataforma / plan	Vercel Hobby (sin tarjeta)

Nota. Elaboración propia a partir de los datos obtenidos del sistema en producción.

7. Discusión

La migración de Render a Vercel constituyó la contingencia más notable: a diferencia de Render, Vercel no exige tarjeta en el plan gratuito y su integración con GitHub reduce el

despliegue a *push* + *import*. La unificación en un único runtime simplifica el modelo mental de despliegue (un solo artefacto, un solo requirements.txt).

La métrica de sostenibilidad es una **estimación** basada en el peso transferido, no una medición directa del consumo eléctrico real; se reporta como tal y queda documentada en el propio endpoint (bloque metodología). La limitación más visible del plan gratuito es el *cold start* (la primera carga tras inactividad es más lenta), comunicada a los usuarios como característica del plan.

El proceso dejó dos aprendizajes de método: **verificar antes de avanzar** (cada fase cierra con pruebas) y **no versionar secretos** (.gitignore + variables de entorno por plataforma).

8. Conclusiones

1. Se logró publicar la plataforma completa en producción sobre Vercel, alcanzando el objetivo de acceso público por HTTPS.
2. El tablero de sostenibilidad aporta una métrica real y replicable de impacto ambiental, integrada con la metodología internacionalmente reconocida de *Website Carbon*.
3. La suite de 33 pruebas e2e garantiza que los flujos críticos (auth, roles, CRUD y métricas) operen correctamente en el despliegue.
4. El plan de gobernanza formaliza seguridad, respaldo, monitorización y respuesta ante incidentes, alineado con OWASP y la Ley N.º 164 de Bolivia.
5. El proyecto quedó en un estado "desplegable en un clic": cualquier *push* a main genera un nuevo despliegue con *rollback* disponible.

9. Recomendaciones

- Migrar a un plan de pago cuando el proyecto lo requiera para eliminar el *cold start* y obtener dominio personalizado, sidetracke no afectando la evaluación académica.

- Incorporar rate-limiting y reCAPTCHA en producción si se prevé tráfico masivo o ataques.
- Automatizar los respaldos manuales semanales con un *job* de GitHub Actions que exporte la base a un bucket.
- Continuar la monitorización de uptime y ampliar el tablero con tendencias históricas de CO₂ evitado.
- Realizar una auditoría de accesibilidad (WCAG) con herramienta automática antes de difundir el sitio.

10. Referencias

- Asamblea Legislativa del Estado Plurinacional de Bolivia. (2011). *Ley N.º 164 de Telecomunicaciones, Tecnologías de Información y Comunicación*. Gaceta Oficial del Estado. <https://www.gacetaoficialdebolivia.gob.bo>
- Flask. (2026). *Flask documentation*. Pallets Projects. <https://flask.palletsprojects.com>
- Greenwood, T., Holmes, R., e Woodruff, C. (2020). *Estimating digital emissions*. Sustainable Web Design. <https://sustainablewebdesign.org/estimating-digital-emissions/>
- OWASP Foundation. (2024). *Application Security Verification Standard (ASVS)*. <https://owasp.org/www-project-application-security-verification-standard/>
- React. (2026). *React documentation*. <https://react.dev>
- Supabase. (2026). *Supabase product documentation*. <https://supabase.com/docs>
- Vercel. (2026). *Framework guide: Flask on Vercel*. <https://vercel.com/docs/frameworks/flask>
- Vite. (2026). *Vite documentation*. <https://vite.dev>
- Website Carbon. (2025). *How does the Website Carbon Calculator work?* <https://www.websitecarbon.com>

Anexo A — Matriz de uso de inteligencia artificial

Herramienta utilizada: asistente de desarrollo *opencode* (modelo big-pickle). La IA colaboró de la siguiente manera, y **cada resultado fue verificado manualmente**:

Tabla 4

Matriz de uso de asistencia de inteligencia artificial y verificación

#	Tarea solicitada a la IA	Salida de la IA	Verificación humana/técnica
1	Preparar despliegue unificado Flask + SPA	Código para servir dist/ con <i>fallback</i> SPA y proxy de desarrollo Vite	Pruebas locales 5/5 contra el servidor real
2	Adaptar el deploy a Vercel (runtime Python)	api/index.py, vercel.json, requirements.txt en raíz	Import local de api/index.py OK; 16 rutas registradas; documentación oficial de Vercel
3	Calcular huella de carbono por vista	Fórmula y blueprint /api/metricas con 3 bloques	contraste contra el peso real de dist/ (342,6 KB → 0,62 g CO ₂ e); revisión del bloque metodología
4	Ampliar pruebas e2e	4 casos nuevos sobre métricas	Ejecución 33/33 PASS
5	Ajustar lint para ignorar el build	ignorePatterns en .oxlintc.json	npm run lint sin advertencias
6	Documentar gobernanza y este informe	Borradores de GOBERNANZA.md e Informe-Actividad-4.md	Revisión de cifras contra la BD real; ajuste de roles/URLs
7	Control de calidad de secretos y git	.gitignore (excluye .env*, __pycache__)	git status confirmó que no se versionan secretos ni cachés

Nota. Elaboración propia a partir de los datos obtenidos del sistema en producción.

Declaración ética: toda sugerencia de la IA fue contrastada con la documentación oficial y probada en el entorno real antes de incorporarse; ninguna credencial ni dato de la base fue divulgado.

Anexo B — Evidencia de despliegue y verificación

- Repositorio público: <https://github.com/DeymarLC/feria-agricultura-familiar>
- Sitio en producción: <https://feria-agricultura-familiar.vercel.app>
- Health check: GET /api/health → {"estado": "ok", ...}
- Swagger UI: /apidocs/ · Especificación: /apispec_1.json (7 rutas)
- Tablero: /sostenibilidad · Métricas: /api/metricas
- Suite de pruebas: backend/scripts/pruebas_e2e.py → Resumen: 33 / 33 pruebas OK
- Historial de despliegue: 8 *commits* en main (Act. 3 y Act. 4) con integración continua en Vercel.

Verificación del sitio publicado (checklist de producción):

Tabla 5

Verificación del sitio publicado (checklist de producción)

Prueba contra https://feria-agricultura-familiar.vercel.app	Resultado
GET /api/health	200 · {"estado":"ok",...}
GET /api/ferias	200 · 3 ferias
GET /api/metricas	200 · 3 bloques (0,62 g CO ₂ e/vista; 3 ferias, 2 productores, 11 productos; 105 km, 16,8 kg CO ₂ e)
GET /, /ingresar, /sostenibilidad	200 · SPA con <i>fallback</i>
GET /apidocs/ y /apispec_1.json	200 · 7 rutas documentadas
Assets JS/CSS	200 (300,9 KB + 29,6 KB)
GET /api/inexistente	404 en formato JSON
Cabecera CORS	Access-Control-Allow-Origin restringido al origen de producción
POST /api/usuarios/login + GET /api/usuarios/me	token JWT; devuelve perfil con rol (admin y productor)
Login con clave incorrecta	401
Smoke test de escritura (crear producto → verificar → eliminar)	201 → visible en catálogo → eliminado de la BD (autolimpieza)

Nota. Elaboración propia a partir de los datos obtenidos del sistema en producción.

Todos los checks del checklist pasaron el 29 de septiembre de 2026.

Documento generado para la evaluación de Programación Web II — Actividad 4.